🎓 AWS з нуля до SAA-C03 — підручник

24 уроки з образами, «запам'ятай», тригерами іспиту і пастками. Інтерактивна версія з підказками, квізом і тестом — на сайті.

Незалежний навчальний курс. Не пов'язаний з Amazon Web Services і не схвалений AWS. AWS і назви сервісів — торговельні марки Amazon.com, Inc. або її афілійованих осіб. Практичні питання — оригінальні, це не реальні питання іспиту.

Уроки

1 🎯 Хто такий Solutions Architect і як влаштований іспит

2 ☁️ Хмара, регіони і зони доступності

3 🤝 Хто за що відповідає і 6 стовпів Well-Architected

4 🪪 IAM — хто і що може робити

5 🔑 Шифрування, ключі й секрети

6 🚨 Охорона й аудит — WAF, Shield, GuardDuty та інші

7 🏘️ VPC — твоє приватне містечко

8 🌉 З'єднання — peering, Transit Gateway, VPN, Direct Connect

9 🧭 DNS і швидкість для всього світу — Route 53, CloudFront, Global Accelerator

10 ⚖️ Балансувальники і автомасштабування

11 🖥️ EC2 — сервери в оренду

12 🧑‍🏭 Serverless і контейнери

13 📨 Черги, події й оркестрація

14 🪣 S3 — безрозмірний склад

15 Диски й файлові системи — EBS, EFS, FSx
16 Гібрид і переїзд у хмару
17 Реляційні бази — RDS і Aurora
18 DynamoDB, кеш і спеціальні бази
19 Аналітика і великі дані
20 Готові AI-сервіси
21 Моніторинг, аудит і автоматизація
22 Висока доступність і відновлення після аварій
23 Вартість — розумний гаманець
24 Як розв'язувати питання іспиту

🎯 Хто такий Solutions Architect і як влаштований іспит

🖼️ **Образ.** Ти — архітектор, який будує місто в хмарі. Замовник каже чотири побажання: «щоб не зламали», «щоб не падало», «щоб було швидко», «щоб недорого». AWS — величезний будівельний магазин на 200+ матеріалів (сервісів). Твоя робота — обрати ті, що виконують **усі** побажання одночасно.

Solutions Architect — людина, яка вирішує, **як** система житиме в AWS: де зберігати дані, як витримати мільйон користувачів, як не втратити дані при аварії і як не переплатити. Іспит SAA-C03 перевіряє саме це вміння **вибирати**.

Майже кожне питання — міні-історія: «Компанія має… Потрібно… Яке рішення найкраще?». Правильна відповідь — не «найкрутіша», а та, що виконує **всі** вимоги з тексту. Часто всі варіанти технічно робочі, але лише один — найдешевший, найпростіший чи найнадійніший саме в цій ситуації.

Чотири побажання замовника — це чотири домени іспиту:

- **Безпека — 30%**: «щоб не зламали».

- **Відмовостійкість — 26%**: «щоб не падало й не губило дані».
- **Продуктивність — 24%**: «щоб було швидко».
- **Вартість — 20%**: «щоб недорого».

Формат: 65 питань (50 оцінюються), 130 хвилин, прохідний бал 720 з 1000. Штрафу за неправильну відповідь немає, тож порожніх не лишай. Якщо англійська не рідна — **до запису** на іспит попроси +30 хвилин (ESL +30).

Як вчитися на цьому сайті — чотири кроки по колу:

- 🎓 **Урок** — зрозуміти й уявити.
- 📘 **Шпаргалка** — стисло повторити факти й тригери.
- 🎲 **Квіз** — закріпити тригери «бачиш у питанні → відповідь».
- 📝 **Тест** — потренуватися як на іспиті.

Увімкни 🤖 **Підказки** вгорі: торкнись будь-якої підкресленої назви сервісу (або виділи її пальцем) — з'явиться коротке пояснення, образ і тригери для іспиту.

💡 **Запам'ятай**

Іспит — про **вибір** сервісу під вимоги, а не про команди й меню консолі.

Шукай у питанні критерій: **MOST cost-effective, LEAST operational overhead, MOST secure, highly available**.

Правильна відповідь виконує **всі** вимоги одночасно.

Відповідай на **всі** питання — штрафу немає.

🎯 **Як питають на іспиті**

«...with the LEAST operational overhead» → **керований або serverless сервіс замість власних серверів і скриптів**

«...the MOST cost-effective solution» → **найдешевше з рішень, що виконують усі вимоги**

Варіант «напишемо скрипт на EC2 і запускатимемо cron» → **майже завжди програє керованому сервісу**

⚠️ **Не плутай**

«Найдешевше» на іспиті ≠ «найдешевше взагалі»: дешевий варіант без потрібної доступності — неправильний.

«Highly available» (працює попри збій, з короткою паузою) ≠ «fault tolerant» (працює взагалі без перерви).

📘 У шпаргалці: 1. Іспит: головні факти · 2. Тактика: як читати питання

Хмара, регіони і зони доступності

🖼️ **Образ.** AWS — мережа «міст» по всьому світу. Кожне місто — **Region** (наприклад, Франкфурт — `eu-central-1`). У кожному місті кілька окремих «районів» зі своїм живленням, охолодженням і мережею — **Availability Zones (AZ)**. А ще біля людей стоять сотні «кіосків» — **edge locations**, звідки швидко роздають контент.

Що таке хмара. Замість купувати сервери і чекати їх місяцями, ти орендуєш потужність на хвилини, платиш за використане і за хвилини збільшуєш або зменшуєш її. Це називають **еластичністю**.

Region обирають з чотирьох причин:

- **Закони** — дані мають лишатися в країні чи в ЄС.
- **Близькість до користувачів** — менша відстань, менша затримка.
- **Потрібні сервіси** — не кожен сервіс є в кожному регіоні.
- **Ціна** — у різних регіонах ціни трохи різні.

AWS не переносить твої дані між регіонами сам — лише коли ти налаштуєш реплікацію чи копіювання.

Availability Zone — один або кілька дата-центрів, ізольованих від аварій в інших AZ, але з'єднаних

швидкою мережею. Звідси головне правило архітектора: **висока доступність = розкласти систему щонайменше у 2 AZ**. Захист від падіння цілого регіону = другий регіон.

Edge locations — точки CloudFront, Route 53 і Global Accelerator: там не запускають сервери, там кешують і приймають трафік ближче до людей.

Особливі «філії» AWS:

- **Local Zones** — AWS у великому місті: одноцифрові мілісекунди до мешканців.
- **Wavelength** — AWS усередині 5G-мережі оператора.
- **Outposts** — стійка AWS у твоєму власному дата-центрі.

Словничок новачка:

- **Сервер / віртуальна машина** — комп'ютер, який ти орендуєш; фізично він в AWS, а керуєш ним через інтернет.
- **Затримка (latency)** — за скільки мілісекунд приходить відповідь.
- **Доступність (availability)** — чи працює сервіс зараз («двері відчинені»).
- **Надійність зберігання (durability)** — чи не загубляться дані («речі на складі цілі»).

💡 **Запам'ятай**

Region ⊃ AZ ⊃ дата-центри.

Збій зони — рятують **2+ AZ**. Збій регіону — рятує **другий регіон**.

Edge location — не регіон: тут кешують, а не запускають сервери.

🎯 **Як питають на іспиті**

Дані мають фізично лишатися в ЄС → **регіон у ЄС + заборона інших регіонів через SCP**

Одноцифрові мілісекунди для мешканців конкретного мегаполіса → **Local Zones**

Дані мають лишатися у своєму дата-центрі, але потрібні сервіси AWS → **Outposts**

Застосунок має пережити збій одного дата-центру → **розгортання у 2+ Availability Zones**

⚠️ **Не плутай**

Availability (працює зараз) ≠ durability (дані не загубляться): 11 дев'яток S3 — це саме **durability**.

Local Zones (місто) ≠ Wavelength (5G) ≠ Outposts (твій дата-центр).

📘 У шпаргалці: 3. Основи: інфраструктура, модель відповідальності, Well-Architected

🤝 Хто за що відповідає і 6 стовпів Well-Architected

🖼️ **Образ.** Оренда квартири. Фундамент, стіни, ліфт і дах — турбота власника будинку (**AWS**: дата-центри, обладнання, мережа, віртуалізація). А хто має ключі від твоїх дверей, що лежить у шафі й чи зачинене вікно — твоя відповідальність (**клієнт**: дані, доступи, шифрування, налаштування).

Це **модель спільної відповідальності (Shared Responsibility Model)**. AWS відповідає за безпеку **хмари** (security OF the cloud), ти — за безпеку **в хмарі** (security IN the cloud).

Чим більше сервіс «керований», тим менше роботи тобі:

- **EC2** — ти сам оновлюєш ОС, ставиш патчі, налаштовуєш Security Groups.
- **RDS** — ОС і рушій бази латає AWS; тобі лишаються доступи, бекап-політики, шифрування.
- **Lambda, S3, DynamoDB** — серверів ти не бачиш зовсім; тобі — дані, права і налаштування.

Але навіть у повністю керованому сервісі дані й доступи — твої. Якщо bucket S3 став публічним, винен не AWS.

Well-Architected Framework — шість «питань інспектора», за якими AWS оцінює будь-яку архітектуру. На них побудований і іспит. Мнемоніка — **CROPS + S**:

- **C**ost Optimization — чи не переплачуємо?
- **R**eliability — чи відновимося після збою?
- **O**perational Excellence — чи автоматизоване керування і чи вчимося на інцидентах?
- **P**erformance Efficiency — чи правильні ресурси під навантаження?
- **S**ecurity — чи захищені дані й доступи?
- **S**ustainability — чи ефективно використовуємо ресурси й енергію?

Кілька принципів, які часто «ховаються» в правильних відповідях: мінімальні права і журнали всіх дій (Security); автоматичне відновлення і горизонтальне масштабування замість одного великого сервера (Reliability); керовані сервіси і serverless (Performance); плати лише за використане і розподіляй витрати тегами (Cost).

💡 **Запам'ятай**

AWS — **OF** the cloud, ти — **IN** the cloud.

EC2: ОС латаєш ти. RDS: ОС латає AWS.

Шість стовпів — **CROPS + S**.

🎯 **Як питають на іспиті**

Хто встановлює патчі ОС на EC2? → **Клієнт**

Хто відповідає за фізичну безпеку дата-центрів? → **AWS**

Хто налаштовує Security Groups і права IAM? → **Клієнт**

Потрібна самоперевірка архітектури за кращими практиками → **AWS Well-Architected Tool**

⚠️ **Не плутай**

Керований сервіс не знімає з тебе відповідальності за **дані і доступи**.

Well-Architected — це рамка кращих практик, а не окремий сервіс захисту.

📘 У шпаргалці: 3. Основи: інфраструктура, модель відповідальності, Well-Architected

🪪 IAM — хто і що може робити

🖼️ **Образ.** Великий офісний центр. **IAM users** — працівники з перепустками. **Groups** — відділи: права дають відділу, а не кожному окремо. **Roles** — тимчасові гостьові бейджі, які охорона (**STS**) видає на кілька годин. **Policies** — таблички на дверях: хто, в які двері і за яких умов може заходити.

Як охорона вирішує, чи пустити — три кроки, які треба знати напам'ять:

1. Ти в **чорному списку** (явний **Deny**)? Не пустить ніхто, навіть якщо десь є дозвіл.
2. Чорного списку немає — охорона шукає **дозвіл (Allow)**.
3. Дозволу немає — **не пускає** (неявна заборона). За замовчуванням заборонено все.

Root user — власник будівлі з майстер-ключем. Увімкни йому MFA, сховай ключ і не користуйся ним щодня.

Ролі замість ключів. Застосунок на EC2 отримує права через роль (instance profile) — це бейдж, прикріплений до робота, який сам оновлюється кожні кілька годин. Access keys у коді чи на сервері — як перепустка, приклеєна скотчем до дверей: рано чи пізно її знайдуть.

Багато акаунтів:

- **AWS Organizations** — головний офіс мережі компаній: один рахунок (consolidated billing), акаунти згруповані в OU.
- **SCP** — правила головного офісу для філій: **стеля**, вище якої ніхто у філії не стрибне, навіть її директор. SCP **нічого не дозволяє сама**, лише обмежує, і не діє на management account.
- **Permission boundary** — стеля для конкретної людини чи ролі (щоб дати розробникам створювати ролі, але не ширші за межу).
- **Крос-акаунт** — роль в акаунті B, у trust policy якої написано, хто з акаунта A може взяти бейдж (sts:AssumeRole). Стороннім компаніям додають умову **sts:ExternalId** — захист від «confused deputy».

Вхід людей:

- **IAM Identity Center** — одна перепустка співробітника на всі акаунти компанії (SSO), можна через корпоративний Okta, Entra ID чи Active Directory.
- **Amazon Cognito** — для **клієнтів застосунку**: *User Pool* — реєстрація і вхід (хто ти), *Identity Pool* — тимчасові AWS-ключі (що тобі можна в AWS), наприклад щоб фото з телефона летіли прямо в S3.

🗺️ Схема: як AWS перевіряє доступ (у PDF «Шпаргалка»)

💡 **Запам'ятай**

Deny б'є все. Потім Allow. Нічого не сказано — заборонено.

SCP — стеля, а не ключ.

Роль — бейдж, а не ключ у кишені.

Співробітники → Identity Center. Клієнти застосунку → Cognito.

🎯 **Як питають на іспиті**

Застосунку на EC2 потрібен доступ до S3 без ключів → **IAM role через instance profile**

Заборонити всім акаунтам в OU вимикати CloudTrail → **SCP**

Розробники створюють ролі, але не ширші за певні права → **Permission boundary**

Користувачі мобільного застосунку завантажують фото прямо в S3 → **Cognito Identity Pool**

SaaS-вендор отримує доступ до акаунта через роль → **Trust policy з умовою sts:ExternalId**

⚠️ **Не плутай**

SCP не діє на management account і не дає прав.

Cognito User Pool (вхід) ≠ Identity Pool (тимчасові AWS-ключі).

Identity Center (працівники) ≠ Cognito (клієнти застосунку).

📘 У шпаргалці: 4. IAM і керування доступом

🔑 Шифрування, ключі й секрети

🖼️ **Образ. KMS** — банк ключів. Ключ тобі на руки не видають: ти приносиш дані й кажеш «зашифруй» або «розшифруй», банк робить це всередині й записує кожну операцію в журнал (CloudTrail). **CloudHSM** — власний сейф у сховищі банку, код від якого знаєш тільки ти.

Дані бувають у двох станах: **at rest** — лежать на диску («сейф удома»), і **in transit** — їдуть мережею («броньована машина», тобто TLS/HTTPS). Захищати треба обидва.

KMS — головне:

- **AWS managed keys** (`aws/s3` , `aws/ebs`) — ключ «від банку за замовчуванням»: ротація щороку, керувати не можна.
- **Customer managed keys** — твій ключ: сам пишеш key policy (хто може ним користуватися), налаштовуєш ротацію, даєш доступ іншим акаунтам.
- **Envelope encryption** — KMS напряму шифрує лише до 4 KB. Великі дані шифрує окремий «ключ даних», а його самого шифрує KMS-ключ: лист у конверті, конверт у сейфі.

- Ключ можна видалити лише з очікуванням 7–30 днів — встигнеш передумати.

S3 шифрує нові об'єкти за замовчуванням (SSE-S3). Потрібен аудит, хто і коли користувався ключем, — обирай **SSE-KMS**. SSE-C — ключ щоразу приносиш сам.

Існуючі диски й бази не можна «зашифрувати галочкою»: знімок → копія знімка з шифруванням → відновлення з копії. Як переїзд у зашифровану квартиру.

Секрети:

- **Secrets Manager** — сейф з автозаміною замків: сам змінює паролі до БД за розкладом (ротація).
- **Parameter Store** — шафка для налаштувань і секретів: дешево, але без вбудованої ротації.

ACM — паспортний стіл для сайтів: безкоштовні TLS-сертифікати для ALB, CloudFront, API Gateway, які поновлюються самі. Для CloudFront сертифікат має бути в регіоні **us-east-1**.

💡 **Запам'ятай**

Аудит використання ключа → **SSE-KMS**. Ключі лише твої, виділене обладнання → **CloudHSM**.

Ротація паролів → **Secrets Manager**. Просто конфіг → **Parameter Store**.

KMS напряму — до **4 KB**, далі envelope encryption.

Зашифрувати існуючу БД чи диск — тільки через **копію знімка**.

🎯 **Як питають на іспиті**

Автоматично змінювати пароль до БД кожні 30 днів → **Secrets Manager**

Ключі лише під контролем компанії, FIPS 140 Level 3, single-tenant → **CloudHSM**

HTTPS на ALB без ручного поновлення сертифіката → **ACM**

ThrottlingException від KMS при масовій роботі з S3 → **S3 Bucket Keys**

Поділитися зашифрованим снапшотом з іншим акаунтом → **Customer managed key + доступ до ключа**

⚠️ **Не плутай**

KMS (керований, спільний) ≠ CloudHSM (виділене залізо, ключі тільки твої).

ACM-сертифікат для CloudFront — лише в us-east-1.

📘 У шпаргалці: 5. Шифрування, ключі, секрети, сертифікати

🚨 Охорона й аудит — WAF, Shield, GuardDuty та інші

🖼️ **Образ.** Торговий центр і його охорона. **WAF** — фейс-контроль на вході: перевіряє кожного відвідувача. **Shield** — дамба від натовпу (DDoS). **GuardDuty** — розумні камери, що помічають дивну поведінку. **Inspector** — технагляд, що шукає тріщини. **Macie** — пес-шукач персональних даних. **Security Hub** — пульт охорони, куди сходяться всі тривоги.

Захист від атак:

- **WAF** (рівень 7, HTTP) — правила проти SQL injection і XSS, блок IP і країн, **rate-based rules** («цей відвідувач заходить надто часто»). Ставиться на CloudFront, ALB, API Gateway, AppSync, Cognito — **але не на NLB**.
- **Shield Standard** — безкоштовно і автоматично для всіх: захист від DDoS рівнів 3–4.
- **Shield Advanced** — платна «страховка»: команда експертів 24/7 і компенсація рахунку, якщо під атакою все масштабувалося.
- **Firewall Manager** — єдиний пульт правил WAF, Shield і Security Groups для всіх акаунтів організації.
- **Network Firewall** — митниця на в'їзді у VPC: фільтрує трафік, зокрема вихідний — за доменами.

Виявлення і розслідування:

- **GuardDuty** — **загрози**: аналізує CloudTrail, VPC Flow Logs і DNS (криптомайнінг, викрадені ключі, підозрілі IP).
- **Inspector** — **вразливості** (CVE) в EC2, образах контейнерів і Lambda.
- **Macie** — **чутливі дані** (PII) в S3.
- **Detective** — слідчий: знаходить першопричину інциденту.
- **Security Hub** — зводить усі знахідки в одне місце і перевіряє стандарти (CIS, PCI DSS).

Журнали і відповідність:

- **CloudTrail** — «хто, що і коли зробив»: кожен виклик API. Хто видалив bucket — питай CloudTrail.
- **Config** — «як виглядав ресурс і чи відповідає правилам»: історія налаштувань і автоматичні перевірки («усі диски зашифровані»).
- **Artifact** — звіти відповідності самого AWS (SOC, PCI) для твоїх аудиторів.

💡 **Запам'ятай**

GuardDuty — охорона від загроз. **Inspect**or — інспектор дефектів. **Macie** — «мої персональні дані».

CloudTrail — **хто** зробив. Config — **що** змінилось і чи за правилами.

WAF — рівень 7 і **не на NLB**.

🎯 **Як питають на іспиті**

SQL injection і XSS на ALB → **AWS WAF**

Обмежити кількість запитів з одного IP → **WAF rate-based rule**

DDoS-експерти 24/7 і компенсація витрат → **Shield Advanced**

Знайти персональні дані в тисячах bucket → **Macie**

Хто змінив Security Group? → **CloudTrail**

Перевіряти, що всі EBS зашифровані, з історією змін → **AWS Config**

⚠️ **Не плутай**

GuardDuty (загрози) ≠ Inspector (вразливості) ≠ Macie (дані).

Shield Standard уже ввімкнений у всіх. Питання про «додатковий захист і команду експертів» — це Shield Advanced.

📘 У шпаргалці: 6. Сервіси безпеки: захист від атак і аудит

🏠 VPC — твоє приватне містечко

🖼️ **Образ. VPC** — огороджене містечко в хмарі. Діапазон адрес (CIDR, напр. `10.0.0.0/16`) — «номери будинків». **Підмережі** — квартали, кожен в одному районі (AZ). **Internet Gateway** — головні ворота на трасу. **Route table** — дорожні вказівники в кожному кварталі.

Публічна чи приватна підмережа — вирішує лише дорожній вказівник. Якщо в route table є маршрут `0.0.0.0/0 → Internet Gateway`, квартал **публічний**. Немає — **приватний**. Зазвичай у публічних підмережах стоять балансувальники, а сервери й бази — у приватних.

NAT Gateway — виїзд з односторонньою смугою: мешканці приватного кварталу можуть виїхати в інтернет (завантажити оновлення), а чужі до них не в'їдуть. Стоїть у публічній підмережі; для надійності — по одному в кожній AZ. Платний за годину і за кожен гігабайт.

Два охоронці — головне питання про мережу на іспиті:

- **Security Group** — охоронець біля дверей конкретного будинку (інстансу). **Пам'ятає**, хто вийшов, і сам пускає відповідь назад (**stateful**). Має лише список «кого пускати» — тільки Allow. Вміє пускати «всіх, у кого бейдж SG вебсерверів».
- **Network ACL** — шлагбаум на в'їзді в квартал (підмережу). **Нічого не пам'ятає** (stateless): перевіряє машини в обидва боки, тому треба окремо дозволити відповіді (порти 1024–65535). Має і дозволи, і **заборони (Deny)**, правила перевіряє по черзі за номером.

VPC endpoints — приватні дороги до сервісів AWS без інтернету:

- **Gateway endpoint** — безкоштовна дорога лише до двох складів: **S3 і DynamoDB**.
- **Interface endpoint** (PrivateLink) — «віконце» з приватною IP-адресою у твоєму кварталі до майже будь-якого сервісу (SQS, KMS, Secrets Manager…). Платний.

Дрібниці, які люблять на іспиті: у кожній підмережі AWS резервує **5 IP-адрес**; розмір VPC — від /16 до /28; адміни заходять на приватні сервери через **SSM Session Manager**, без відкритого порту 22 і без бастіону.

🗺️ Схема: типова VPC (у PDF «Шпаргалка»)

💡 Запам'ятай

Публічна підмережа = є маршрут на Internet Gateway.

SG — Stateful, лише Allow. NACL — No memory (stateless), Numbered, може Deny.

Gateway endpoint — тільки «SD»: **S**3 і **D**ynamoDB.

NAT — лише назовні; для високої доступності — по одному на AZ.

🎯 Як питають на іспиті

Сервери в приватній підмережі мають завантажувати оновлення → **NAT Gateway у публічній підмережі**

Заблокувати конкретну IP-адресу → **Network ACL з правилом Deny**

Приватний EC2 звертається до S3 без інтернету і безкоштовно → **Gateway endpoint**

Приватний EC2 звертається до SQS або KMS без інтернету → **Interface endpoint**

База приймає трафік лише від вебсерверів → **SG бази з джерелом = SG вебсерверів**

⚠️ **Не плутай**

У Security Group немає Deny — блокувати IP можна лише NACL (або WAF).

Один NAT Gateway на всю VPC — єдина точка відмови і платний трафік між AZ.

📘 У шпаргалці: 7. VPC і мережа

🌉 З'єднання — peering, Transit Gateway, VPN, Direct Connect

🖼️ **Образ.** Твої VPC — окремі містечка. **VPC Peering** — міст між двома містечками. **Transit Gateway** — велика транспортна розв'язка, де сходяться дороги всіх містечок і офісу. **Site-to-Site VPN** — броньований фургон по загальній трасі (інтернету). **Direct Connect** — власна приватна залізниця до AWS.

VPC Peering:

- Зв'язує дві VPC — навіть у різних акаунтах і регіонах.
- **Не транзитивний**: A–B і B–C не дають A–C. Кожній парі — свій міст.
- Адреси (CIDR) не можуть перетинатися — інакше «листи заблукають».
- Дешевий і простий для 2–3 VPC.

Transit Gateway:

- Хаб для десятків і сотень VPC, VPN і Direct Connect, маршрутизація **транзитивна**.
- Ділиться між акаунтами через AWS RAM, між регіонами — через TGW peering.
- Платиш за кожне підключення і за кожен гігабайт трафіку.

PrivateLink — «віконце видачі»: сусіди отримують твій сервіс (через NLB), не заходячи в твоє містечко. CIDR можуть збігатися, а споживачів — тисячі.

З'єднання з офісом:

- **Site-to-Site VPN** — зашифровані IPsec-тунелі через інтернет. Налаштовується за хвилини, недорого, 2 тунелі для резерву, до 1,25 Gbps на тунель.
- **Direct Connect** — виділений приватний канал 1–400 Gbps: стабільна швидкість, дешевший вихідний трафік, але прокладання займає **тижні**. Сам по собі **не шифрує** — для шифрування VPN поверх нього або MACsec. Резерв — друга лінія в іншій локації або VPN.
- **Client VPN** — для ноутбуків окремих працівників.
- **Гібридний DNS** — Route 53 Resolver endpoints: inbound (офіс питає імена AWS) і outbound (VPC питає імена офісу).

🗺️ Схема: гібридне з'єднання (у PDF «Шпаргалка»)

💡 **Запам'ятай**

2–3 VPC → **Peering**. Багато VPC і офіс → **Transit Gateway**.

Швидко й дешево → **VPN**. Стабільно й великі обсяги → **Direct Connect**.

Direct Connect **не шифрує** сам.

🎯 **Як питають на іспиті**

З'єднати десятки VPC і офіс з транзитивною маршрутизацією → **Transit Gateway**

Надати сервіс сотням клієнтських VPC, адреси перетинаються → **PrivateLink**

Канал до офісу потрібен уже сьогодні → **Site-to-Site VPN**

Direct Connect + вимога шифрування → **VPN поверх Direct Connect або MACsec**

Недорогий резерв для Direct Connect → **Site-to-Site VPN**

⚠️ **Не плутай**

Peering нетранзитивний і не працює з однаковими CIDR.

VPN — через інтернет (шифрування є, стабільності немає). Direct Connect — приватний (стабільно, але без шифрування за замовчуванням).

📘 У шпаргалці: 7. VPC і мережа

🧭 DNS і швидкість для всього світу — Route 53, CloudFront, Global Accelerator

🖼️ **Образ. Route 53** — довідкова інтернету: перекладає ім'я (example.com) в адресу і ще вирішує, **яку** адресу дати саме тобі. **CloudFront** — мережа місцевих магазинів з копіями товару: береш у найближчому, а не їдеш на центральний склад. **Global Accelerator** — VIP-коридор в аеропорту з двома постійними входами: заходиш у найближчі двері, далі йдеш швидкою приватною доріжкою AWS.

Route 53 — як довідкова обирає відповідь (routing policies):

- **Simple** — один номер.
- **Weighted** — «90% дзвінків у старий офіс, 10% — у новий» (поступовий реліз).
- **Latency** — до регіону з найменшою затримкою для користувача.
- **Failover** — основний, а якщо перевірка здоров'я провалилась — запасний.
- **Geolocation** — за країною користувача («за паспортом»).
- **Geoproximity** — за відстанню, з можливістю зсунути трафік (bias).

- **Multivalue** — кілька живих адрес на вибір.

Alias — ярлик на ресурс AWS (ALB, CloudFront, S3): працює й для кореневого домену example.com, де звичайний CNAME заборонений, і такі запити безкоштовні.

CloudFront:

- Кешує статичний контент на edge-локаціях і прискорює динамічний через мережу AWS.
- **OAC** — склад S3 відчиняється тільки кур'єрам CloudFront: bucket лишається приватним.
- **Signed URL** — квиток на один файл; **signed cookies** — абонемент на багато файлів.
- Блокування країн, WAF, Shield, HTTPS (сертифікат ACM у us-east-1).

Global Accelerator:

- 2 статичні **anycast IP** для всього світу; протоколи TCP і **UDP**; миттєве перемикання між регіонами без очікування DNS-кешу.
- **Нічого не кешує** — це не CDN.

🗺️ Схема: високодоступний застосунок (у PDF «Шпаргалка»)

💡 **Запам'ятай**

CloudFront — Кешує Контент (HTTP). Global Accelerator — Гоночна доріжка + статичні IP (TCP/UDP).

Alias — для кореня домену і безкоштовний для ресурсів AWS.

Поступово — weighted; найшвидше — latency; запасний — failover; за країною — geolocation.

🎯 **Як питають на іспиті**

Зменшити затримку для користувачів в Азії, статика + динаміка → **CloudFront**

Гра на UDP, статичні IP, швидке перемикання регіонів → **Global Accelerator**

Закрити S3 від прямого доступу, лише через CloudFront → **OAC**

Пустити 10% трафіку на нову версію → **Route 53 weighted routing**

Active-passive між регіонами через DNS → **Route 53 failover + health checks**

⚠️ **Не плутай**

CloudFront кешує, Global Accelerator — ні.

Geolocation (країна) ≠ latency (швидкість) ≠ geoproximity (відстань + bias).

📘 У шпаргалці: 8. DNS і доставка контенту: Route 53, CloudFront, Global Accelerator

⚖️ Балансувальники і автомасштабування

🖼️ **Образ.** Супермаркет. **Балансувальник** — адміністратор біля входу, що розводить покупців по касах. **Auto Scaling** — менеджер зміни: черги ростуть — відкриває нові каси, спад — закриває зайві, касир захворів — ставить заміну.

Три балансувальники:

- **ALB** (рівень 7, HTTP/HTTPS) — адміністратор, що **читає, куди тобі треба**: маршрутизує за шляхом (`/api`, `/images`), доменом, заголовками; вміє відправляти запити в Lambda; тримає кілька сертифікатів (SNI); пускає через Cognito. Статичного IP немає.
- **NLB** (рівень 4, TCP/UDP) — **швидкісний турнікет**: мільйони запитів, наднизька затримка, **статичний IP** (можна Elastic IP), основа для PrivateLink.
- **GWLB** (рівень 3) — **пункт огляду багажу**: пропускає весь трафік через сторонні файрволи чи IDS і масштабує їх.

Auto Scaling group (ASG):

- Тримає min / desired / max інстансів у кількох AZ і замінює нездорові.

- **Увімкни ELB health checks** — інакше ASG бачить лише, що «сервер увімкнений», а не що застосунок відповідає.
- Як масштабувати: **target tracking** («тримай CPU біля 50%» — найпростіше), **step**, **scheduled** («щоп'ятниці о 18:00 більше кас»), **predictive** («прогноз за історією»).
- Воркерів, що обробляють чергу, масштабуй за кількістю повідомлень у SQS на один інстанс.
- Корисне: lifecycle hooks (дія перед стартом чи видаленням), warm pools (прогріті інстанси), instance refresh (оновити всіх на новий AMI).

Stateless — золоте правило: касир не тримає твій кошик. Сесії зберігаються в ElastiCache або DynamoDB, а не на сервері — тоді сервери можна вільно додавати й прибирати.

💡 **Запам'ятай**

ALB — «читає адресу» (L7). NLB — «швидкий турнікет зі статичною IP» (L4). GWLB — «огляд багажу» (L3).

ASG + **ELB health checks**.

Відомий час піку → scheduled. Циклічні піки → predictive. Тримати метрику → target tracking.

🎯 Як питають на іспиті

`/api` і `/images` на різні групи серверів → **ALB path-based routing**

Клієнт додає IP балансувальника у whitelist → **NLB з Elastic IP (або Global Accelerator)**

Весь трафік через сторонній файрвол → **Gateway Load Balancer**

ASG не замінює сервер, де застосунок завис → **ELB health checks в ASG**

Пік щопонеділка о 9:00 → **Scheduled scaling**

⚠️ Не плутай

ALB не має статичного IP.

Sticky sessions — милиця. Правильно — зберігати сесії поза сервером.

📘 У шпаргалці: 9. Балансувальники (ELB) і Auto Scaling

🖥️ EC2 — сервери в оренду

🖼️ **Образ.** EC2 — автопрокат комп'ютерів. **Тип інстансу** — модель авто: **C** — спорткар (потужний процесор), **R** — вантажівка з великим кузовом (багато пам'яті), **I** — авто з величезним швидким багажником (диски), **P/G** — з турбіною (GPU), **T/M** — універсал на кожен день.

Як читати назву: `m7g.2xlarge` — сімейство **m**, покоління **7**, **g** — процесор Graviton (ARM, помітно вигідніший), розмір **2xlarge**.

Як платити — найважливіша тема для питань про вартість:

🚕 **On-Demand**

Таксі: платиш за час, без зобов'язань

Короткі й непередбачувані задачі

$ $ $

🚙 **Savings Plans / RI**

Абонемент або лізинг на 1–3 роки, до ~72% знижки

Стабільне навантаження 24/7

$

🎟️ **Spot**

Горящий тур: до 90% дешевше, але можуть забрати з попередженням за 2 хв

Задачі, які можна переривати

$ найдешевше

🏠 **Dedicated Host**

Цілий гараж тільки твій: видно ядра й сокети

Ліцензії BYOL, комплаєнс

$ $ $ $

Compute Savings Plans діють на будь-яке сімейство і регіон, а також на Fargate і Lambda. Capacity Reservation — бронь місця в AZ без знижки.

Розміщення (placement groups):

- **Cluster** — команда за одним столом: максимальна швидкість обміну між серверами (HPC), але одна «пожежа» зачепить усіх.
- **Spread** — кожен у своєму будинку: максимум 7 інстансів на AZ, для кількох критичних серверів.
- **Partition** — секції з окремими входами: сотні серверів Hadoop, Kafka, Cassandra.

Диски і стан:

- **EBS** — «підключений жорсткий диск», дані переживають вимкнення.

- **Instance store** — записи крейдою на дошці: найшвидше, але зникає при зупинці.
- **AMI** — «зліпок» комп'ютера для швидкого клонування; регіональний, копіюється в інші регіони.
- **Hibernate** — сон ноутбука: вміст пам'яті зберігається на диск.
- **User data** — скрипт при першому запуску; **IMDSv2** — безпечний доступ до метаданих інстансу.

💡 **Запам'ятай**

Spot — **2 хвилини** попередження. Spread — **7 на AZ**.

Постійно — Savings Plans/RI. Можна переривати — Spot. Невідомо — On-Demand. Ліцензії на ядра — Dedicated Host.

Instance store зникає, EBS — ні.

🎯 **Як питають на іспиті**

Нічні пакетні задачі, які можна перезапускати, найдешевше → **Spot Instances**

24/7 на 3 роки + свобода міняти сімейства і перейти на Fargate → **Compute Savings Plans**

Ліцензії Windows або SQL Server прив'язані до фізичних ядер → **Dedicated Hosts**

HPC з мінімальною затримкою між вузлами → **Cluster placement group + EFA**

Кодування відео (навантаження на процесор) → **C-сімейство**

⚠️ **Не плутай**

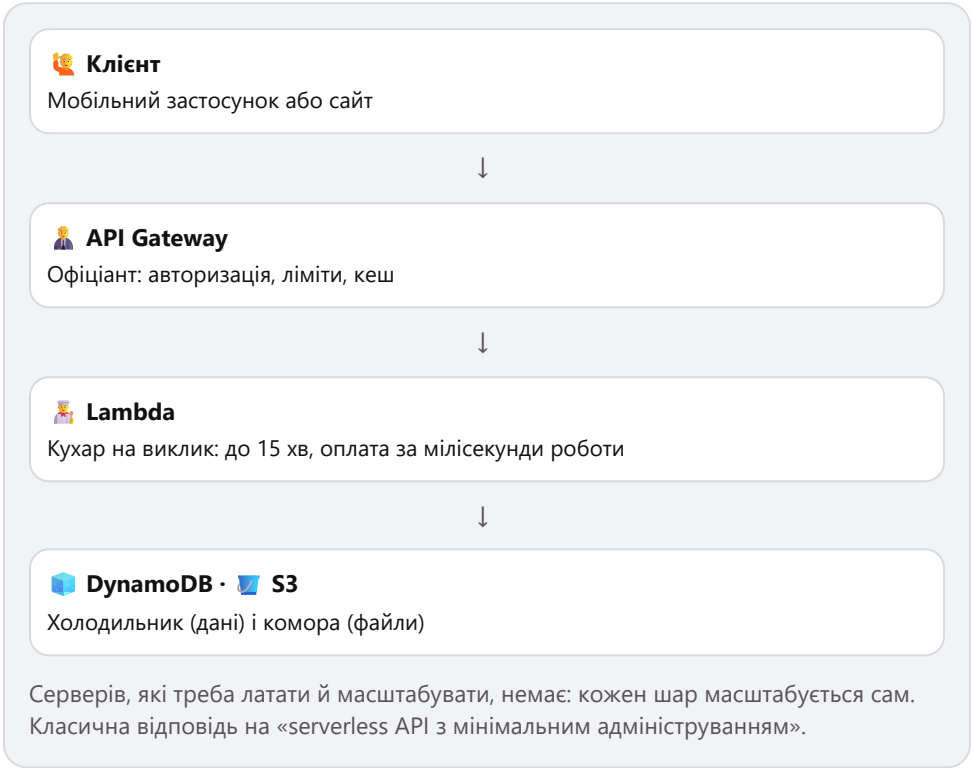
Spot не для основної бази і не для задач без чекпоінтів.

Вертикальне масштабування (більший інстанс) — з зупинкою. Горизонтальне (більше інстансів) — через ASG.

📘 У шпаргалці: 10. EC2: інстанси, тарифи, розміщення

👨‍🍳 Serverless і контейнери

🖼️ **Образ.** Ресторан без власної кухні. **API Gateway** — офіціант: приймає замовлення, перевіряє, чи ти гість, стежить, щоб не було натовпу. **Lambda** — кухар на виклик: приходить, готує страву (до 15 хвилин), і ти платиш лише за приготоване. **DynamoDB** — холодильник, **S3** — комора.



Серверів, які треба латати й масштабувати, немає: кожен шар масштабується сам. Класична відповідь на «serverless API з мінімальним адмініструванням».

Lambda — головне:

- До **15 хвилин** на виклик, пам'ять 128 MB – 10 GB (процесор росте разом з пам'яттю), 1 000 одночасних виконань за замовчуванням.
- Запускається подіями: новий файл у S3, повідомлення в SQS, запит з API Gateway, розклад EventBridge, зміни в DynamoDB.
- **Cold start** — кухар щойно прийшов і розкладає ножі. Ліки: **provisioned concurrency** (кухарі вже чекають на кухні) і **SnapStart** (Java, Python, .NET).
- Забагато з'єднань до RDS → **RDS Proxy**. Обмежити функцію → reserved concurrency.
- Довше 15 хвилин → Step Functions (розбити на кроки), ECS/Fargate або AWS Batch.

Контейнери — стандартні «морські контейнери» для застосунків: усередині програма з усім потрібним, і вона однаково працює будь-де.

- **ECS** — диспетчер порту від AWS (простіше). **EKS** — керований Kubernetes (якщо команда вже на K8s).
- **Fargate** — порт сам дає крани й причали: контейнери без серверів (для ECS і EKS).
- **ECR** — склад образів контейнерів.

- **Elastic Beanstalk** — ресторан під ключ: приносиш код, AWS сам ставить сервери, балансувальник і масштабування.
- **AWS Batch** — цех для великих пакетних задач будь-якої тривалості.

💡 **Запам'ятай**

Lambda: 15 хвилин, 10 GB, 1 000 одночасно.

Cold start → provisioned concurrency або SnapStart.

Контейнери без серверів → Fargate. Уже є Kubernetes → EKS.

🎯 **Як питають на іспиті**

Задача виконується 40 хвилин → **не Lambda: AWS Batch, ECS/Fargate або Step Functions**

Serverless REST API → **API Gateway + Lambda + DynamoDB**

Lambda вичерпує з'єднання до бази → **RDS Proxy**

Розробники хочуть просто задеплоїти вебзастосунок → **Elastic Beanstalk**

Різні ліміти запитів для різних клієнтів API → **API Gateway usage plans**

⚠️ **Не плутай**

Beanstalk не serverless — він створює EC2, просто керує ними за тебе.

Reserved concurrency (ліміт) ≠ provisioned concurrency (прогріті середовища).

📘 У шпаргалці: 11. Serverless і контейнери

Черги, події й оркестрація

🖼️ **Образ.** Поліклініка. **SQS** — талончики в черзі: пацієнти (повідомлення) чекають, а лікарі (воркери) беруть по одному у своєму темпі. **SNS** — гучномовець: одне оголошення чують усі одразу. **EventBridge** — диспетчерська з правилами «яка подія — куди». **Step Functions** — головлікар з покроковим планом лікування.

SQS (pull — «забирай сам»):

- Розв'язує компоненти й згладжує піки: замовлення не губляться, навіть якщо бекенд упав (повідомлення зберігаються до 14 днів).
- **Visibility timeout** — талон у руках лікаря: інші його не бачать. Не встиг за відведений час — талон повертається в чергу, і його обробить хтось іще. Ліки від «обробки двічі» — збільшити timeout.
- **Standard** — майже без лімітів, але можливі дублі й зміна порядку. **FIFO** — строгий порядок і без дублів.
- **DLQ** — окрема папка для «безнадійних» талонів після кількох невдалих спроб.
- Long polling — лікар чекає до 20 секунд, а не смикає реєстратуру щосекунди (дешевше).

SNS (push — «кажу всім»):

- Одне повідомлення → багато підписників: SQS, Lambda, email, SMS, HTTP.
- SNS — не черга: «на потім» нічого не зберігає. Тому класика — **fan-out**: SNS → кілька SQS, у кожного обробника своя поштова скринька.

EventBridge — події від AWS, твоїх застосунків і SaaS (Zendesk, Shopify…) → правила → цілі. Плюс **Scheduler** — будильник (cron) без серверів.

Step Functions — покроковий процес з повторами, розгалуженнями і паузою на ручне схвалення. Standard — до року; Express — до 5 хвилин, для величезних потоків.

Kinesis Data Streams — річка даних з пам'яттю: кілька заводів беруть воду з тієї ж річки в реальному часі й можуть «перемотати» назад. **Data Firehose** — трубопровід, що сам доставляє потік у S3, Redshift чи OpenSearch.

Amazon MQ — перекладач для старих застосунків на JMS, AMQP чи MQTT (RabbitMQ, ActiveMQ) без переписування коду.

🗺️ Схема: fan-out SNS → SQS (у PDF «Шпаргалка»)

💡 **Запам'ятай**

SQS — Запитай сам (pull). SNS — Скажи всім (push).

Обробка двічі → visibility timeout. «Отруйні» повідомлення → DLQ. Порядок без дублів → FIFO.

Потік з перемоткою → Kinesis Data Streams. Просто доставити в S3 → Firehose.

🎯 **Як питають на іспиті**

Замовлення губляться під час піків → **SQS між компонентами**

Одна подія — три незалежні обробники → **SNS fan-out → три черги SQS**

Фінансові транзакції строго по порядку → **SQS FIFO**

Процес з ручним схваленням на кілька днів → **Step Functions Standard**

Запускати задачу щоночі о 2:00 → **EventBridge Scheduler**

⚠️ **Не плутай**

SQS не видаляє повідомлення сам — споживач робить DeleteMessage після обробки.

SNS — не черга: без SQS повідомлення можуть пропасти.

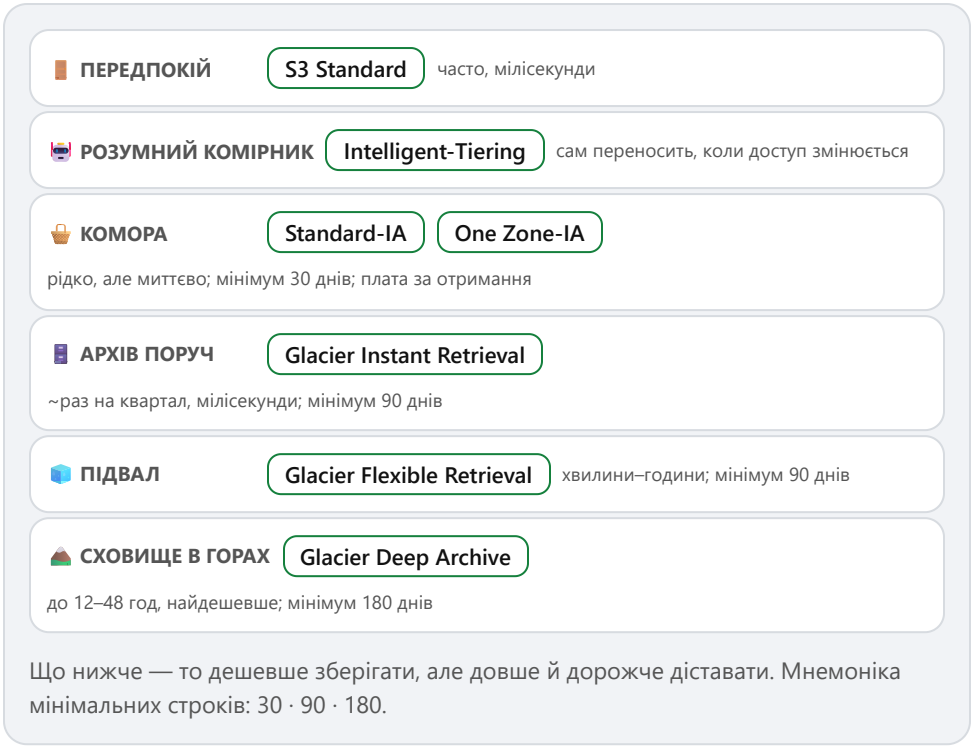
📘 У шпаргалці: 12. Інтеграція і повідомлення: SQS, SNS, EventBridge, Step Functions, MQ

🪣 S3 — безрозмірний склад

🖼️ **Образ.** S3 — безрозмірний склад. **Bucket** — окремий склад з унікальною на весь світ назвою. **Object** — коробка з ярликом-адресою (ключем), розміром до 50 ТВ. «Повно» на цьому складі не буває, а надійність — 11 дев'яток: з 10 мільйонів коробок у середньому губиться одна за 10 000 років.

S3 — не диск і не файлова система, а сховище об'єктів, до якого звертаються через HTTP/API. Ідеальне для файлів, фото й відео, бекапів, логів, data lake і статичних сайтів. Змонтувати S3 як диск для бази не можна.

Класи зберігання — будинок від передпокою до сховища в горах:



Керування складом:

- **Lifecycle** — автоматичне переселення речей з віком: через 30 днів у комору, через 90 — в архів, через 7 років — викинути.
- **Versioning** — машина часу: перезапис і видалення не знищують попередніх версій.
- **Object Lock** — сейф з таймером (WORM): у режимі compliance не видалить ніхто, навіть root; у governance — лише з особливим дозволом.

- **Replication** — філія складу в іншому регіоні (CRR) або в тому самому (SRR); потрібен versioning.

Доступ і безпека: Block Public Access увімкнено за замовчуванням; права — через bucket policy та IAM; **presigned URL** — тимчасовий пропуск на одну коробку; сайт з HTTPS — через CloudFront + OAC.

Швидкість: 3 500 записів і 5 500 читань за секунду **на кожен префікс** (більше префіксів — швидше); великі файли — **multipart upload** частинами; з далеких країн — **Transfer Acceleration**.

🗺️ Схема: lifecycle (у PDF «Шпаргалка»)

💡 **Запам'ятай**

30 · 90 · 180 — мінімальні строки для IA · Glacier · Deep Archive.

Невідомий патерн доступу → **Intelligent-Tiering**.

WORM на роки → **Object Lock, compliance mode**.

🎯 **Як питають на іспиті**

Дані читають рідко, але потрібні миттєво → **S3 Standard-IA**

Зберігати 7–10 років для регулятора, доступ до 48 год → **Glacier Deep Archive**

Ніхто, навіть root, не може видалити записи 7 років → **Object Lock compliance mode**

Копія кожного нового об'єкта в іншому регіоні → **Cross-Region Replication**

Тимчасово дати клієнту скачати приватний файл → **Presigned URL**

⚠️ **Не плутай**

One Zone-IA живе в одній AZ — лише для даних, які можна відтворити.

Видалив з IA чи Glacier раніше мінімального строку — однаково заплатиш за весь строк.

📘 У шпаргалці: 13. Amazon S3 і Glacier

💿 Диски й файлові системи — EBS, EFS, FSx

🖼️ **Образ.** Три способи тримати документи. **Блочне сховище (EBS)** — жорсткий диск, прикручений до одного комп'ютера. **Файлове (EFS, FSx)** — спільна мережева папка для всього офісу. **Об'єктне (S3)** — склад, куди звертаються через інтернет.

EBS:

- Живе в **одній AZ** і підключається до одного інстансу (виняток — Multi-Attach для io1/io2).
- **Snapshot** — «фотографія» диска в S3: з неї створюють диск в іншій AZ чи регіоні.
- Типи: **gp3** — універсал, продуктивність налаштовується окремо від розміру (до 80 000 IOPS); **io2 Block Express** — гоночний SSD для критичних баз (до 256 000 IOPS, затримка менше мілісекунди); **st1** — вантажівка для великих послідовних даних (логи, big data); **sc1** — холодний і найдешевший. HDD-диски (st1, sc1) не можуть бути системними.
- Шифрування за замовчуванням вмикається однією галочкою на весь регіон.

Instance store — крейда на дошці: найшвидше, але зникає при зупинці. Лише для кешу й тимчасових даних.

EFS — спільна мережева папка (NFS) для **Linux**: тисячі серверів у кількох AZ одночасно, об'єм росте сам. Класи Standard, Infrequent Access і Archive з автоматичним переселенням. Security Group має пропускати порт 2049.

FSx — спеціалізовані файлові сервери:

- **for Windows File Server** — SMB, NTFS і Active Directory для Windows-програм.
- **for Lustre** — болід Формули-1 для HPC і машинного навчання: сотні GB/s, дружить із S3.
- **for NetApp ONTAP** — швейцарський ніж: NFS, SMB і iSCSI одночасно.
- **for OpenZFS** — для переїзду з ZFS.

💡 **Запам'ятай**

Блок — один сервер (EBS). Файли — багато серверів (EFS для Linux, FSx для Windows і HPC). Об'єкти — через HTTP (S3).

Понад 80 000 IOPS на одному томі → **io2 Block Express**.

Windows + AD → **FSx for Windows**. HPC + S3 → **FSx for Lustre**.

🎯 Як питають на іспиті

Спільна папка для сотень Linux-серверів у різних AZ → **EFS**

Спільні файли для Windows з Active Directory → **FSx for Windows File Server**

HPC з даними в S3 → **FSx for Lustre**

Дешевше за gp2 і без простою → **перейти на gp3**

Перенести диск в іншу AZ → **snapshot → новий том у потрібній AZ**

⚠️ Не плутай

EBS не можна підключити до інстансу в іншій AZ.

EFS — лише Linux (NFS); для Windows — FSx for Windows.

📘 У шпаргалці: 14. Блочне і файлове сховище: EBS, Instance Store, EFS, FSx

🚚 Гібрид і переїзд у хмару

🖼️ **Образ.** Переїзд великої компанії в хмару. **Storage Gateway** — перехідник між старим офісом і хмарою. **DataSync** — швидкісна вантажівка по трасі (мережею). **Snowball** — фізичний контейнер, який везуть поштою, коли траса надто повільна. **DMS** — переїзд магазину без зачинення: товари переносять, а продажі тривають.

Гібридне зберігання — Storage Gateway:

- **S3 File Gateway** — звичні NFS/SMB-папки в офісі, а файли лягають у S3 (з локальним кешем).
- **Volume Gateway** — диски iSCSI: *Cached* — основні дані в хмарі, *Stored* — основні локально, копія в хмарі.
- **Tape Gateway** — віртуальні касети замість справжніх для наявного бекап-ПЗ.

Перенесення даних:

- **DataSync** — онлайн-перенесення і синхронізація терабайтів з NAS, NFS, SMB чи HDFS у S3, EFS або FSx — з перевіркою цілісності й за розкладом.
- **Transfer Family** — SFTP/FTPS/FTP-сервер поверх S3: партнери надсилають файли як і раніше.

- **Snowball Edge** — офлайн-перенесення сотень терабайтів; орієнтир — якщо мережею довше тижня. (3 листопада 2025 недоступний новим клієнтам, але на іспиті може зустрітися.)

Бази даних:

- **DMS** — міграція з мінімальним простоєм: стара база працює, а зміни безперервно копіюються (CDC).
- Однаковий рушій (MySQL → Aurora MySQL) — достатньо DMS. Різні рушії (Oracle → Aurora PostgreSQL) — спершу **SCT** («перекладач креслень»), потім DMS.

Сервери:

- **Application Migration Service (MGN)** — «перевезти будинок цілком»: lift-and-shift серверів у EC2.
- **Elastic Disaster Recovery** — резервна копія серверів в AWS з RPO в секунди.

7 R — стратегії переїзду: Retire (викинути), **Retain** (залишити), **Rehost** (перевезти як є — MGN), **Relocate** (разом з фундаментом — VMware Cloud on AWS), **Replatform** (перевезти й трохи підлатати, напр. базу на RDS), **Repurchase** (купити готовий SaaS), **Refactor** (перебудувати по-новому, serverless).

💡 **Запам'ятай**

Жити гібридно → **Storage Gateway**. Перенести мережею → **DataSync**. Без мережі → **Snowball**.

Різні рушії бази → **SCT + DMS**.

Сервери «як є» → **MGN**.

🎯 **Як питають на іспиті**

Офісні програми пишуть файли в S3 через NFS/SMB → **S3 File Gateway**

Замінити фізичні стрічки, лишивши бекап-ПЗ → **Tape Gateway**

Перенести 50 ТВ з NAS мережею з перевіркою → **DataSync**

Oracle → Aurora PostgreSQL з мінімальним простоєм → **SCT + DMS**

Партнери надсилають файли по SFTP, а зберігати треба в S3 → **Transfer Family**

⚠️ **Не плутай**

DataSync (перенести) ≠ Storage Gateway (жити гібридно) ≠ Transfer Family (протоколи SFTP/FTP).

📘 У шпаргалці: 15. Гібрид, міграція і перенесення даних

🛢️ Реляційні бази — RDS і Aurora

🖼️ **Образ. RDS** — найнятий адміністратор бази даних: сам встановлює, латає, робить бекапи. **Multi-AZ** — дублер у запасній кімнаті: синхронно записує все, але з клієнтами не говорить; основний знепритомнів — дублер за хвилину-дві стає до роботи. **Read replicas** — помічники, які лише відповідають на запитання (читання) і можуть трохи відставати.

RDS:

- MySQL, PostgreSQL, MariaDB, Oracle, SQL Server, Db2. Доступу до операційної системи немає (крім RDS Custom).
- **Multi-AZ** — для **виживання**: синхронна копія в іншій AZ, автоматичне перемикання (зазвичай 60–120 с), та сама адреса підключення.
- **Read replicas** — для **читання**: асинхронні, до 15, можуть бути в іншому регіоні (і стати основною базою під час аварії). Застосунок сам надсилає читання на репліку.
- Бекапи на 1–35 днів + відновлення на будь-яку мить (point-in-time). Відновлення завжди створює **нову** базу з новою адресою.

- **RDS Proxy** — секретар, що тримає телефонні лінії: пул з'єднань для тисяч функцій Lambda і швидше перемикання.

Aurora — «турбо-версія» MySQL і PostgreSQL від AWS:

- 6 копій даних у 3 AZ, сховище саме росте до 256 TiB.
- До 15 реплік на спільному сховищі, перемикання приблизно за 30 с, репліки можна додавати автоматично.
- Входи: **writer** (запис), **reader** (читання, розподіляє між репліками), **custom** (окремий вхід, напр. для аналітиків).
- **Global Database** — філії в інших регіонах (до 10) з відставанням менше секунди; перемикання менш ніж за хвилину.
- **Serverless v2** — «авто з автоматичною коробкою»: сам додає потужність і може «заснути» до нуля, тоді платиш лише за сховище.
- **Backtrack** — перемотка назад без відновлення з бекапу (лише Aurora MySQL). **Cloning** — миттєва копія для тестів. **Babelfish** — розуміє мову SQL Server (T-SQL).

💡 **Запам'ятай**

Multi-AZ — для виживання. Read Replica — для читання.

Aurora: **«6 копій у 3 AZ»**, 15 реплік, Global Database — менше секунди.

Lambda + RDS → **RDS Proxy**.

🎯 **Як питають на іспиті**

Звіти аналітиків гальмують основну базу → **Read replica**

Автоматичне перемикання бази в іншу AZ → **RDS Multi-AZ**

Реляційна база переживе падіння регіону, RPO ~1 с → **Aurora Global Database**

База простоює вночі й на вихідних → **Aurora Serverless v2**

Переїзд з SQL Server без переписування T-SQL → **Babelfish for Aurora PostgreSQL**

⚠️ **Не плутай**

Standby у Multi-AZ (звичайний інстанс) не читається.

Відновлення зі знімка — це нова база з новою адресою.

📘 У шпаргалці: 16. Бази даних: Amazon RDS · 17. Amazon Aurora

🗃️ DynamoDB, кеш і спеціальні бази

🖼️ **Образ. DynamoDB** — гігантська картотека: знаєш номер картки (ключ) — отримуєш її за мілісекунди, навіть коли запитів мільйони. **DAX** — кишенька з найчастішими картками (мікросекунди). **ElastiCache** — записник на столі: не треба щоразу бігти в архів (у базу).

DynamoDB:

- Serverless NoSQL: мілісекунди на будь-якому масштабі, дані в кількох AZ автоматично. Одна «картка» (елемент) — до 400 KB.
- Ключ — **partition key** (номер ящика) + за потреби sort key. Обирай ключ з багатьма різними значеннями (userId), інакше буде «гарячий» ящик і throttling.
- Оплата: **on-demand** (за кожен запит — для непередбачуваного трафіку) або **provisioned** (абонемент на RCU/WCU + auto scaling — для стабільного).
- **Global Tables** — картотека в кількох регіонах із записом у будь-якому.

- **Streams** — стрічка змін → Lambda. **TTL** — картки самі знищуються з часом. **PITR** — відновлення на будь-яку секунду за 35 днів.
- Складних JOIN немає — це не реляційна база.

Кеш:

- **ElastiCache (Redis OSS / Valkey)** — реплікація, Multi-AZ, складні структури: sorted sets для лідербордів, сесії.
- **Memcached** — простий швидкий кеш без реплікації.
- Кеш вимагає змін у коді («спершу глянь у записник»). DAX — майже без змін, але тільки для DynamoDB.

Спеціальні бази — «кожній задачі своя»:

- **Neptune** — карта знайомств (графи: «друзі друзів», шахрайські зв'язки).
- **DocumentDB** — сумісна з MongoDB. **Keyspaces** — з Cassandra. **Timestream** — щоденник показників за часом (IoT).
- **Redshift** — аналітичне сховище (наступний урок). **OpenSearch** — пошуковик по твоїх даних.

💡 **Запам'ятай**

Ключ-значення, мілісекунди, serverless → DynamoDB. Мікросекунди для DynamoDB → DAX.

Лідерборд → Redis/Valkey sorted sets.

Граф → Neptune. MongoDB → DocumentDB. Cassandra → Keyspaces. Часові ряди → Timestream.

🎯 Як питають на іспиті

Кошики, сесії, профілі гравців на мільйонах запитів → **DynamoDB**

Запис з низькою затримкою у двох регіонах → **DynamoDB Global Tables**

Реагувати на кожну зміну в таблиці → **DynamoDB Streams + Lambda**

Автоматично видаляти сесії через 24 год → **TTL**

Лідерборд у реальному часі → **ElastiCache for Redis/Valkey**

⚠️ Не плутай

DAX — лише для DynamoDB; для RDS — ElastiCache.

«Без змін коду» — не ElastiCache: йому потрібна логіка кешування в застосунку.

📘 У шпаргалці: 18. Amazon DynamoDB · 19. Кешування та інші бази даних

📊 Аналітика і великі дані

🖼️ **Образ. Data lake** — озеро, куди стікають усі річки даних у сирому вигляді (S3). **Glue** — бібліотекар: складає каталог, що де лежить, і наводить лад (ETL). **Athena** — питаєш SQL прямо в озера. **Redshift** — склад з упорядкованими полицями для звітів. **Quick** — вітрина з дашбордами для керівників.

Потоки:

- **Kinesis Data Streams** — річка в реальному часі: кілька споживачів, перемотка (replay) до 365 днів.
- **Data Firehose** — трубопровід: сам складає потік у S3, Redshift чи OpenSearch і може перетворити JSON у Parquet.
- **Managed Service for Apache Flink** — аналіз потоку «на льоту» (вікна часу, аномалії).
- **MSK** — керований Kafka для наявних Kafka-застосунків.

Озеро і запити:

- **Glue** — crawler визначає схему, **Data Catalog** зберігає опис таблиць, ETL перетворює CSV у Parquet.

- **Athena** — SQL по файлах у S3 без серверів; платиш за прочитані дані. CSV — щоб знайти одне слово, читаєш усю книгу; **Parquet з партиціями** — одразу відкриваєш потрібну главу. Тому в рази дешевше і швидше.
- **Redshift** — сховище даних для важкої регулярної аналітики; **Spectrum** — «зі складу заглянути в озеро»; zero-ETL з Aurora.
- **EMR** — важкий завод Spark/Hadoop (task-вузли на Spot — дешево).
- **OpenSearch** — пошук і аналіз логів.
- **Amazon Quick** (колишній QuickSight) — дашборди.
- **Lake Formation** — охорона озера: доступ аж до окремих колонок і рядків.

🗺️ Схема: data lake (у PDF «Шпаргалка»)

💡 **Запам'ятай**

Реальний час + перемотка → **Kinesis Data Streams**. Просто доставити → **Firehose**.

Разові SQL по S3 → **Athena**. Регулярна важка аналітика → **Redshift**.

Дешевше в Athena → **Parquet + стиснення + партиції**.

🎯 Як питають на іспиті

Кілька застосунків читають клікстрім у реальному часі → **Kinesis Data Streams**

Складати потік у S3 як Parquet без коду → **Data Firehose**

SQL по логах у S3 без серверів → **Athena**

Права до окремих колонок і рядків у data lake → **Lake Formation**

Дашборди для бізнесу → **Amazon Quick**

⚠️ Не плутай

Firehose не має перемотки і власних споживачів.

Athena — не сховище: дані лишаються в S3.

📘 У шпаргалці: 20. Аналітика і потоки даних

🤖 Готові AI-сервіси

🖼️ **Образ.** Набір готових «органів чуття» для твого застосунку. **Rekognition** — очі. **Transcribe** — вуха. **Polly** — голос. **Translate** — перекладач. **Comprehend** — читач, що розуміє настрій і зміст тексту. **Textract** — бухгалтер, що читає документи й таблиці. **Lex** — співрозмовник-чатбот. **SageMaker AI** — лабораторія, де вирощують власний «мозок» (модель).

Головна думка для іспиту: якщо задача стандартна (розпізнати обличчя, перетворити мову в текст, перекласти) — бери **готовий сервіс**, знання машинного навчання не потрібні. Якщо потрібна власна модель — **SageMaker AI**.

- **Rekognition** — об'єкти й обличчя на фото і відео, модерація контенту.
- **Textract** — текст, **форми й таблиці** зі сканів і PDF (більше, ніж простий OCR).
- **Comprehend** — тональність відгуків, ключові фрази, мова, персональні дані в тексті.
- **Transcribe** — мова → текст (дзвінки, субтитри).
- **Polly** — текст → мова (озвучення). Мнемоніка: папуга Поллі говорить.

- **Translate** — машинний переклад.
- **Lex** — чатботи (голос і текст).
- **SageMaker AI** — навчити, розгорнути й обслуговувати свою модель.

Ланцюжок, який люблять на іспиті: записи дзвінків → **Transcribe** → текст → **Comprehend** (настрій) або **Translate** → **Polly** (озвучити).

А ще для фронтенду: **Amplify** — хостинг і бекенд для веб- і мобільних застосунків, **Device Farm** — тестування на сотнях справжніх телефонів.

💡 **Запам'ятай**

Очі — Rekognition. Вуха — Transcribe. Голос — Polly. Документи — Textract. Розуміння тексту — Comprehend.

Власна модель — **SageMaker AI**.

🎯 **Як питають на іспиті**

Модерація фото користувачів → **Rekognition**

Дані з рахунків і форм у PDF → **Textract**

Тональність відгуків → **Comprehend**

Субтитри з відео → **Transcribe**

Озвучити статті → **Polly**

⚠️ **Не плутай**

Transcribe (мова → текст) ≠ Polly (текст → мова).

Textract (документи, форми) ≠ Rekognition (фото і відео).

📘 У шпаргалці: 21. Машинне навчання, AI і фронтенд

📈 Моніторинг, аудит і автоматизація

🖼️ **Образ.** Автомобіль. **CloudWatch** — приладова панель: спідометр (метрики), лампочки (аларми), бортовий журнал (логи). **CloudTrail** — відеореєстратор: записує кожну дію водія (виклик API). **Config** — фотоархів: як виглядала машина вчора і чи відповідає правилам техогляду. **X-Ray** — рентген поїздки запиту через усі вузли.

CloudWatch:

- Метрики EC2: процесор, мережа, диск, статус. **Пам'ять і вільне місце на диску він не бачить** без **CloudWatch agent** — датчик пального ставлять окремо.
- Аларми → сповіщення (SNS), масштабування, дії з EC2 (відновити, перезавантажити).
- Logs: зберігання й пошук логів, metric filters («порахуй рядки з ERROR»), передача далі в реальному часі.

CloudTrail — хто, що, коли і звідки зробив через API. Історія за 90 днів безкоштовна; для довгого зберігання — trail у S3; organization trail — одразу всі акаунти.

Config — історія налаштувань ресурсів і правила відповідності з автовиправленням: «усі bucket приватні»,

«усі диски зашифровані».

Systems Manager — пульт керування автопарком серверів:

- **Session Manager** — зайти на сервер без SSH і відкритих портів;
- **Run Command** — виконати команду на всіх серверах одразу;
- **Patch Manager** — встановлювати патчі за розкладом;
- **Parameter Store** — налаштування і секрети.

CloudFormation — креслення будинку (інфраструктура як код): за одним кресленням будують однакові середовища будь-де. **StackSets** — одразу в багатьох акаунтах і регіонах.

Порадники: Trusted Advisor (вартість, безпека, ліміти сервісів), **Compute Optimizer** (правильний розмір інстансів), **Health Dashboard** (збої й обслуговування AWS, які зачіпають саме тебе).

💡 **Запам'ятай**

CloudWatch — Watch (здоров'я і метрики). CloudTrail — Trail (сліди дій). Config — стан і правила.

Пам'ять на EC2 → **CloudWatch agent**.

Без SSH → **Session Manager**.

🎯 Як питають на іспиті

Моніторити пам'ять і диск EC2 → **CloudWatch agent**

Хто видалив ресурс → **CloudTrail**

Автоматично перевіряти й виправляти налаштування → **Config rules + remediation**

Де гальмує запит між мікросервісами → **X-Ray**

Однакова інфраструктура в багатьох акаунтах → **CloudFormation StackSets**

⚠️ Не плутай

CloudTrail ≠ CloudWatch: сліди дій проти метрик і логів.

CloudWatch не бачить пам'ять без агента.

📘 У шпаргалці: 22. Моніторинг, керування, інфраструктура як код

🛟 Висока доступність і відновлення після аварій

🖼️ **Образ. Висока доступність** — запасне колесо і другий водій: щось зламалось — їдемо далі після короткої зупинки. **Відмовостійкість** — двомоторний літак: один мотор відмовив, а політ триває без зупинки. **Disaster recovery (DR)** — план на випадок пожежі в цілому місті (регіоні).

Два числа, які визначають усе:

- **RPO** — скільки даних можна втратити: «скільки останніх сторінок щоденника не шкода».
- **RTO** — скільки часу можна простояти: «через скільки ресторан знову годує гостей».

Стратегії DR — від дешевої до дорогої:

- **Backup & Restore** — фото документів у сейфі: після аварії відновлюєш усе з бекапів. RPO і RTO — години.
- **Pilot Light** — запальник котла: у другому регіоні постійно «горить» лише база (реплікується), решту розпалюєш під час аварії. Десятки хвилин.
- **Warm Standby** — прогріта запасна кухня з мінімальним персоналом: зменшена копія працює завжди, при аварії її збільшують. Хвилини.

- **Multi-site Active-Active** — два ресторани працюють одночасно. Майже нуль, але найдорожче.

Цеглинки надійності:

- 2+ AZ всюди: ALB + Auto Scaling + Multi-AZ база; сервери без збереженого стану.
- Черги SQS між компонентами: збій одного не валить інших.
- Другий регіон: Route 53 failover або Global Accelerator; Aurora Global Database; DynamoDB Global Tables; S3 Cross-Region Replication; AMI і знімки в другому регіоні; CloudFormation, щоб швидко відтворити інфраструктуру.
- **AWS Backup** — централізовані бекапи з копіями між регіонами й акаунтами; **Vault Lock** — навіть адмін не видалить бекап (захист від ransomware).
- Старий застосунок на одному сервері → Auto Scaling group з min = max = 1: зламався — його замінять автоматично.

🗺️ Схема: стратегії DR (у PDF «Шпаргалка») · 🗺️ Схема: високодоступний застосунок (у PDF «Шпаргалка»)

💡 **Запам'ятай**

RPO — дані (скільки втратити). RTO — час (скільки стояти).

Від дешевої до дорогої: **Backup** → **Pilot Light** → **Warm Standby** → **Active-Active**.

Обирай найдешевшу стратегію, що вкладається в RPO і RTO з питання.

🎯 **Як питають на іспиті**

RTO десятки хвилин, база реплікується, сервери запускаються при аварії → **Pilot Light**

RTO хвилини, зменшена копія працює постійно → **Warm Standby**

Бекапи не може видалити навіть адміністратор → **AWS Backup Vault Lock**

Один сервер має автоматично відновлюватися після збою → **Auto Scaling group з min = max = 1**

Реляційна база з RPO ~1 с між регіонами → **Aurora Global Database**

⚠️ **Не плутай**

Multi-AZ рятує від збою зони, але не від збою регіону.

Бекап ≠ висока доступність: з бекапу відновлюються годинами.

📘 У шпаргалці: 23. Висока доступність і Disaster Recovery

💰 Вартість — розумний гаманець

🖼️ **Образ.** В'їзд у місто AWS безкоштовний, а виїзд — платний. Переїзд між районами (AZ) — платний міст. **NAT Gateway** — платний КПП, що бере гроші з кожної тонни вантажу (гігабайта). **Gateway endpoint** — безкоштовна приватна дорога до S3 і DynamoDB. **CloudFront** — доставка з місцевих складів, дешевша за доставку з центру.

Принцип: плати лише за використане і підбирай спосіб оплати під характер навантаження.

- **Обчислення:** постійне навантаження → Savings Plans або RI; переривне → Spot; невідоме → On-Demand або serverless; правильний розмір → Compute Optimizer; процесори Graviton; вимикати dev/test на ніч (EventBridge Scheduler).
- **Сховище:** lifecycle у S3, Intelligent-Tiering, gp3 замість gp2, прибирати старі знімки й непідключені диски.
- **Бази:** Aurora Serverless v2 або DynamoDB on-demand для нерівномірного навантаження; кеш замість більшого сервера; Database Savings Plans.

- **Мережа:** gateway endpoints замість NAT для S3 і DynamoDB; CloudFront для роздачі контенту; для кількох VPC з великим трафіком peering дешевший за Transit Gateway; Direct Connect для великих постійних обсягів.

Інструменти:

- **Budgets** — ліміт на картці з SMS: сповіщення, коли витрати або прогноз перевищать поріг.
- **Cost Explorer** — виписка з графіками, прогнозом і порадами щодо Savings Plans.
- **Cost and Usage Report** — найдетальніший чек (кожен ресурс, кожна година) для аналізу в Athena.
- **Cost allocation tags** — ярлики на чеках: розподіл витрат між відділами (теги треба активувати в Billing).
- **Cost Anomaly Detection** — «банк помітив дивну покупку».

💡 **Запам'ятай**

Постійне — абонемент (Savings Plans/RI). Переривне — Spot. Невідоме — On-Demand або serverless.

В'їзд безкоштовний; виїзд і «міст» між AZ — платні.

Сповіщення про бюджет → **Budgets**. Аналіз → **Cost Explorer**. Деталі → **CUR**.

🎯 Як питають на іспиті

Великий рахунок за NAT через трафік до S3 → **Gateway endpoint для S3**

Сповіщення, коли прогноз витрат перевищить $1000 → **AWS Budgets**

Розподілити витрати між відділами → **Cost allocation tags**

Невідомий патерн доступу до файлів → **S3 Intelligent-Tiering**

Dev-сервери потрібні лише в робочі години → **Зупиняти за розкладом**

⚠️ Не плутай

«Найдешевше» на іспиті — найдешевше з рішень, що виконують **усі** вимоги.

Трафік між AZ платний навіть в одній VPC.

📘 У шпаргалці: 24. Оптимізація витрат

🏁 Як розв'язувати питання іспиту

🖼️ **Образ.** Кожне питання — детектив. Замовник залишив у тексті докази (вимоги). Твоє завдання — знайти ключові докази, викреслити «підозрюваних з алібі» (неможливі варіанти) і обрати того, хто підходить під **усі** докази.

Алгоритм на кожне питання (≈2 хвилини):

1. Прочитай **останнє речення** — там критерій: MOST cost-effective, LEAST operational overhead, MOST secure, highly available.
2. Виділи вимоги: регіон чи AZ, RPO/RTO, «без змін коду», real-time, serverless, шифрування.
3. Викресли неможливе: Deny у Security Group, WAF на NLB, Lambda на годину, EBS в іншій AZ, читання з Multi-AZ standby.
4. З решти обери керований і простіший варіант, що виконує **всі** вимоги.
5. Є сумнів — познач (flag), дай найкращу відповідь і повернися в кінці. Порожніх не лишай.

Три розбори:

- **«Замовлення губляться під час непередбачуваних піків. Як розв'язати компоненти?»** Шукаємо буфер, що зберігає повідомлення → **SQS**. SNS не зберігає, Kinesis — для потокової аналітики, більший сервер не лікує піки.
- **«EC2 у приватній підмережі пишуть у S3; мінімальна вартість, без інтернету».** Потрібен приватний шлях до S3 без плати за гігабайти → **gateway endpoint**. NAT Gateway бере гроші за трафік, interface endpoint платний, публічна підмережа — гірша безпека.
- **«Дані читають рідко, але коли треба — миттєво; зберігати рік».** Рідко + мілісекунди → **S3 Standard-IA** (якщо раз на квартал — Glacier Instant Retrieval). Glacier Flexible — хвилини чи години, One Zone-IA — ризик втратити дані разом з AZ.

Фінальна підготовка: план на 14 днів — у шпаргалці (розділ 29). Проходь «Тест» у режимі «Іспит»; якщо стабільно 80%+ — можна записуватися. І не забудь подати **ESL +30 хвилин** до запису.

💡 **Запам'ятай**

Останнє речення → вимоги → викреслити неможливе → керований варіант, що виконує все.

Штрафу немає — відповідай на все.

🎯 Як питають на іспиті

«LEAST operational overhead» → **керований або serverless сервіс**

«without code changes» → **RDS Proxy, Storage Gateway, EFS, Global Accelerator, MGN**

«real-time with multiple consumers» → **Kinesis Data Streams**

«decouple» / «buffer» → **SQS**

⚠️ Не плутай

Надто складна відповідь з купою сервісів і скриптів — зазвичай пастка, якщо є простий керований шлях.

📘 У шпаргалці: 2. Тактика: як читати питання · 26. «X чи Y?» — найчастіші плутанини · 27. Пастки, на яких найчастіше ловлять